

What is HTML?

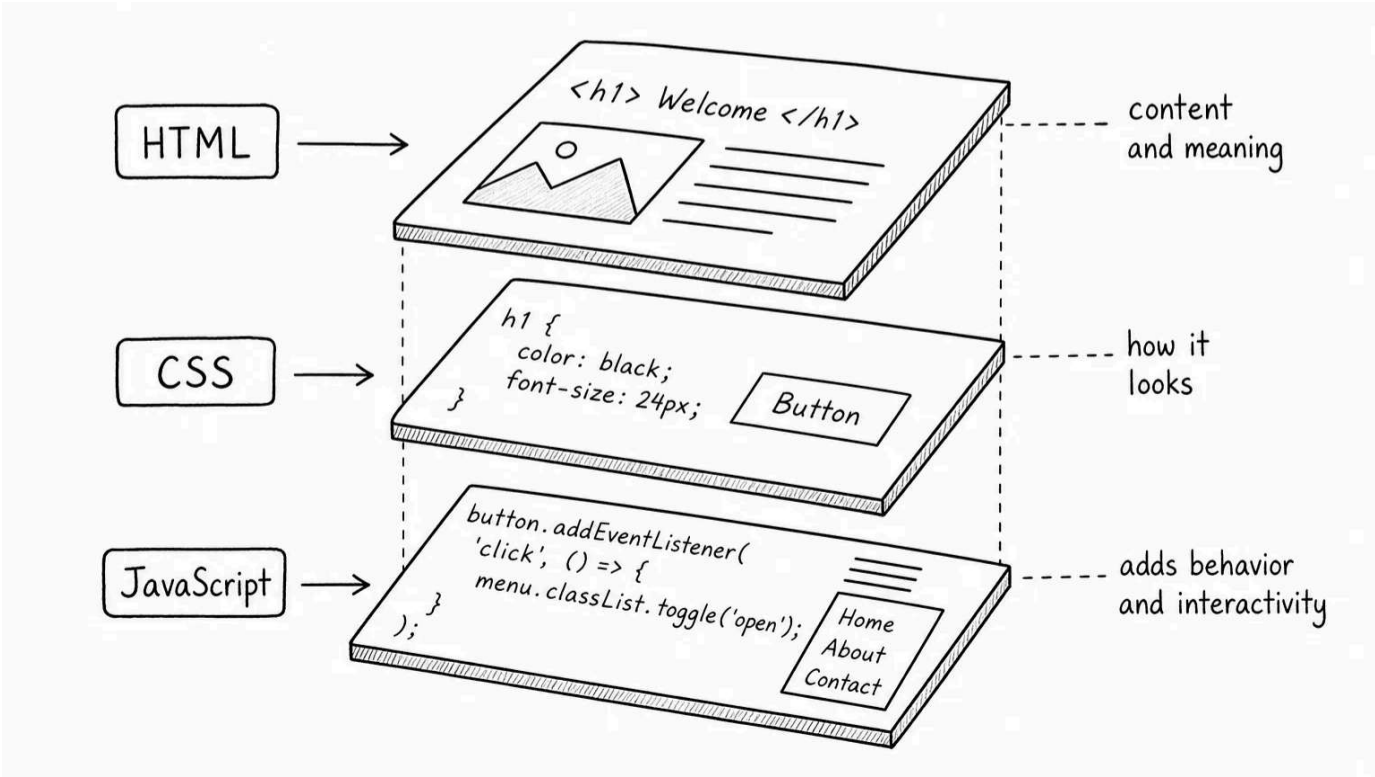
In the [browser lesson](#), you saw that when you visit a webpage, the browser's first HTTP request usually asks for an HTML document. The server sends back HTML, and the browser uses that HTML as the starting point for rendering the page.

HTML gives a webpage its content and structure. Before a page can be styled, animated, or made interactive, the browser needs HTML to know what is on the page.

At a high level, a webpage is built from a few pieces working together:

- **The server** sends HTML to the browser.
- **HTML** describes the content and meaning.
- **CSS** controls how that content looks.
- **JavaScript** adds behavior, like opening menus, reacting to clicks, validating forms, or loading new data without a full page refresh.

The details come later in the pack. For now, hold onto the basic picture: HTML is the starting structure. Everything else either produces it, styles it, or changes the DOM that the browser builds from it.



HTML stands for **HyperText Markup Language**. "Hypertext" means text that can link to other documents. The more important part for this lesson is "markup."

HTML is markup, not programming

HTML is not a programming language. It has no variables, no loops, no functions, no logic. You cannot write a calculator in pure HTML.

What HTML does is *describe*. It is plain text with tags that label parts of a document: this is a heading, this is a paragraph, this is a link, this is an image. The browser reads those labels and uses them to build the page.

A very small HTML file looks like this:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>My first page</title>
  </head>
  <body>
    <h1>Hello, web</h1>
    <p>This is a paragraph of text.</p>
  </body>
</html>
```

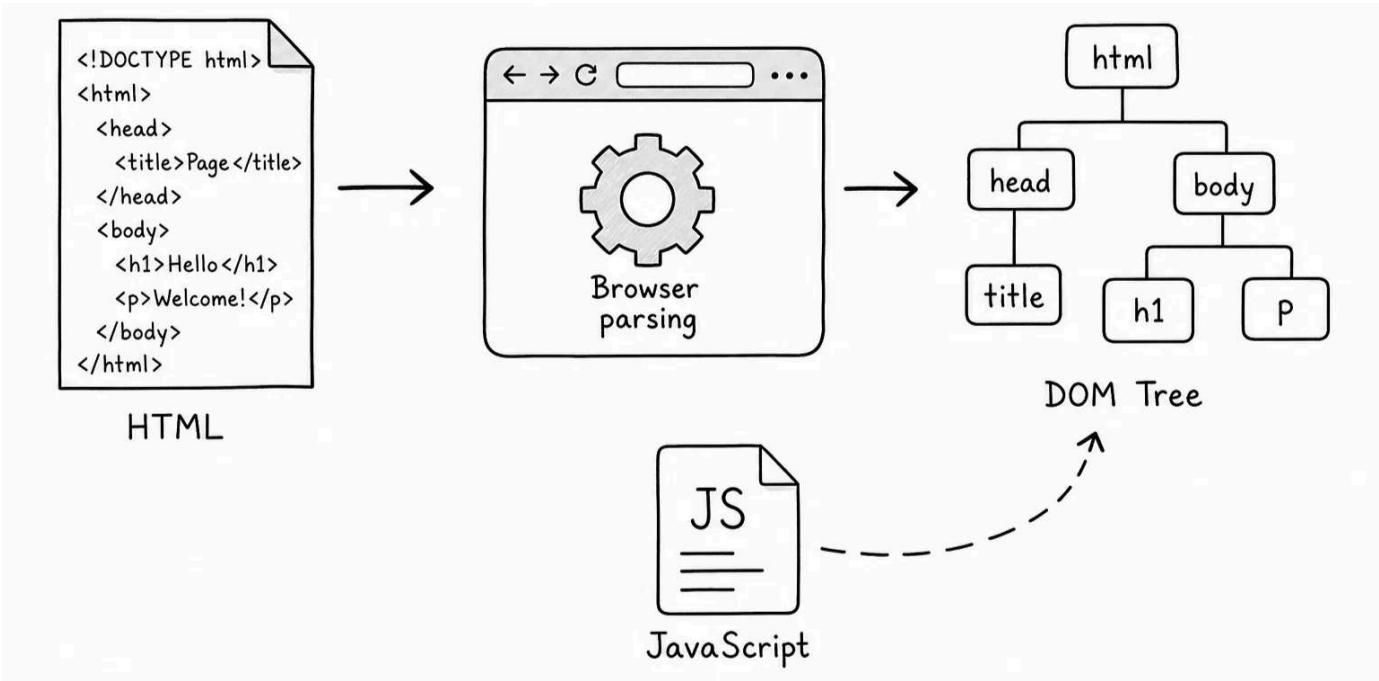
If you save that as `hello.html` and open it in a browser, you get a page with a heading and a paragraph. Nothing has been compiled. No server is running. The browser read the text in that file and figured out what to draw.

You will learn the syntax in the next lesson: what tags are, how attributes work, why some tags do not need a closing tag, and what the lines inside `head` are doing. For now, the important idea is that an HTML file is a description of structure and meaning, not a program.

HTML becomes the DOM

When the browser receives HTML, it does not just print the text on the screen. It parses the file into a tree-shaped structure called the **DOM**, short for Document Object Model.

Each element in the HTML file becomes a node in the tree. The `html` element contains the `head` and the `body`. The `body` contains headings, paragraphs, lists, images, and so on. Nested elements become nested nodes.



The DOM matters because it is what everything else on the page works with. CSS uses the DOM to find elements and apply styles. JavaScript uses the DOM to read content, react to clicks, and change what is on screen. Accessibility tools, search engines, and browser extensions look at the DOM too, not the raw HTML text.

So HTML is the input, and the DOM is what the browser actually works with.

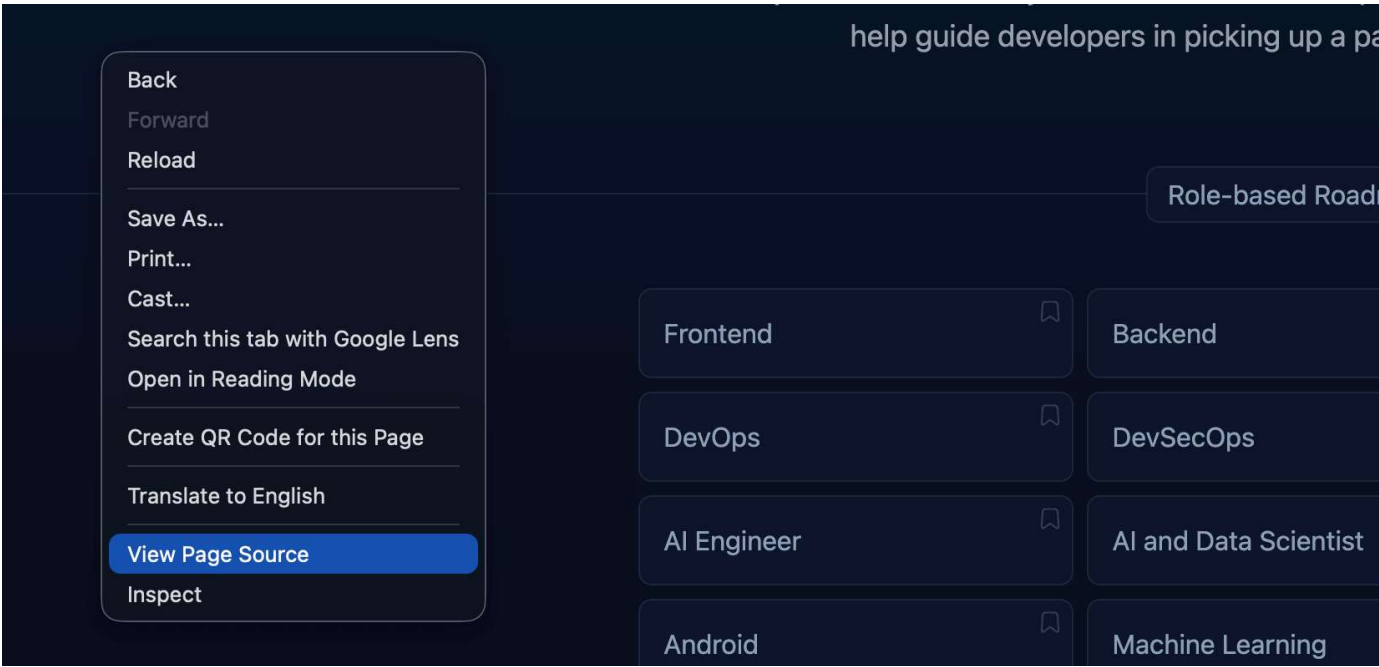
The DOM can change after the page loads

This is the detail that trips beginners up the most.

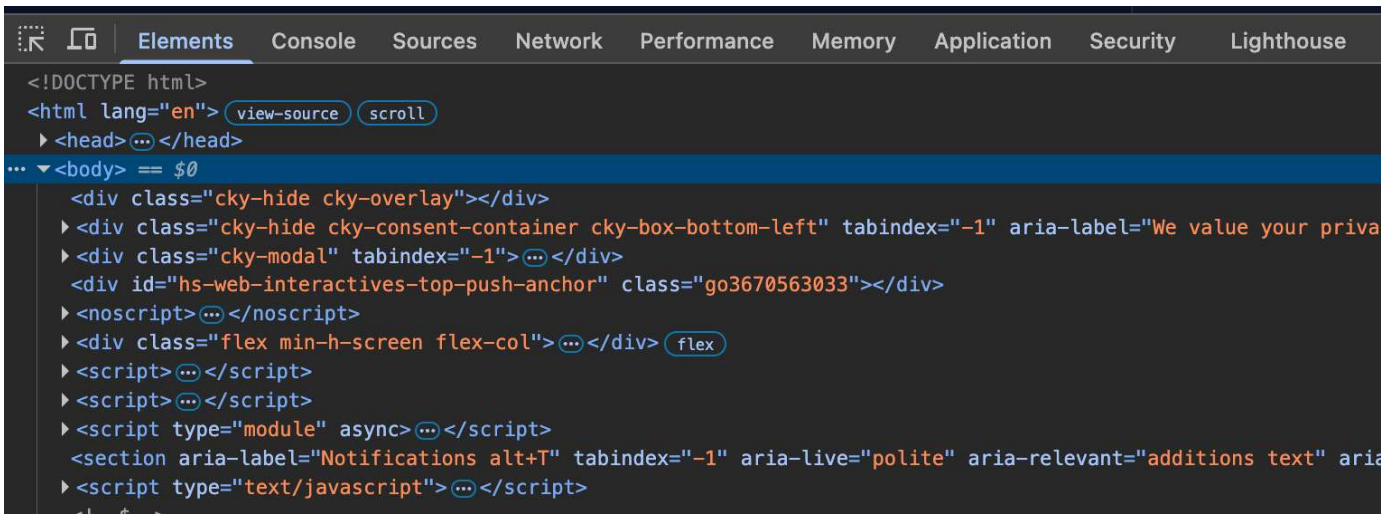
The DOM usually starts from the HTML the server sent. But once the page is loaded, JavaScript can add, remove, or change nodes in the tree. By the time you are looking at a page, the DOM may not match the original HTML response anymore.

That difference shows up in your browser's developer tools:

View Page Source shows the *original* HTML the server sent. It does not change. You can access this by right-clicking on a webpage and choosing **View Page Source**.



Elements in DevTools shows the *current* DOM. It updates as JavaScript runs.



On a simple static page, the two will look almost identical. On a modern web app, View Source might show a mostly empty HTML shell, while Elements shows a fully built-out page. That is JavaScript filling in the DOM after the initial response.

⚠️ **The DOM is not the file** - The DOM starts from the HTML response, but it is a live structure inside the browser, not a copy of the file. Confusing the two will make a lot of frontend behavior look like magic later.

You do not need to think about this every time you read HTML. Just remember that "the page" the browser shows is the DOM, not the HTML text by itself.

Where HTML shows up in real work

HTML shows up across frontend, backend, and fullstack work.

Frontend developers write HTML directly or through components. Fullstack and backend developers often produce HTML through templates, where server data is mixed with markup before being sent to the browser. Server frameworks also render HTML emails, error pages, admin dashboards, and confirmation screens.

The HTML your app sends or generates is what the browser parses, what users see, what screen readers announce, and what search engines index. It is part of the product, not a leftover detail.



Show me a small example of a server-rendered HTML page where the same template is filled with different data for two different users. Keep it beginner friendly.

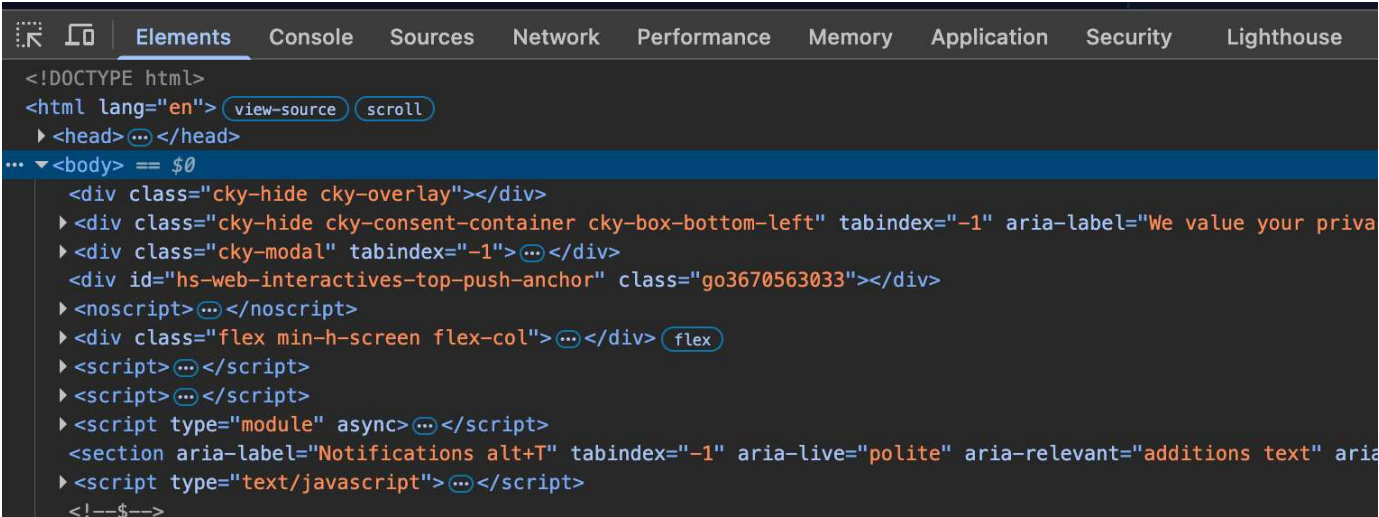
Try it

Take the following HTML and save it as `hello.html` :

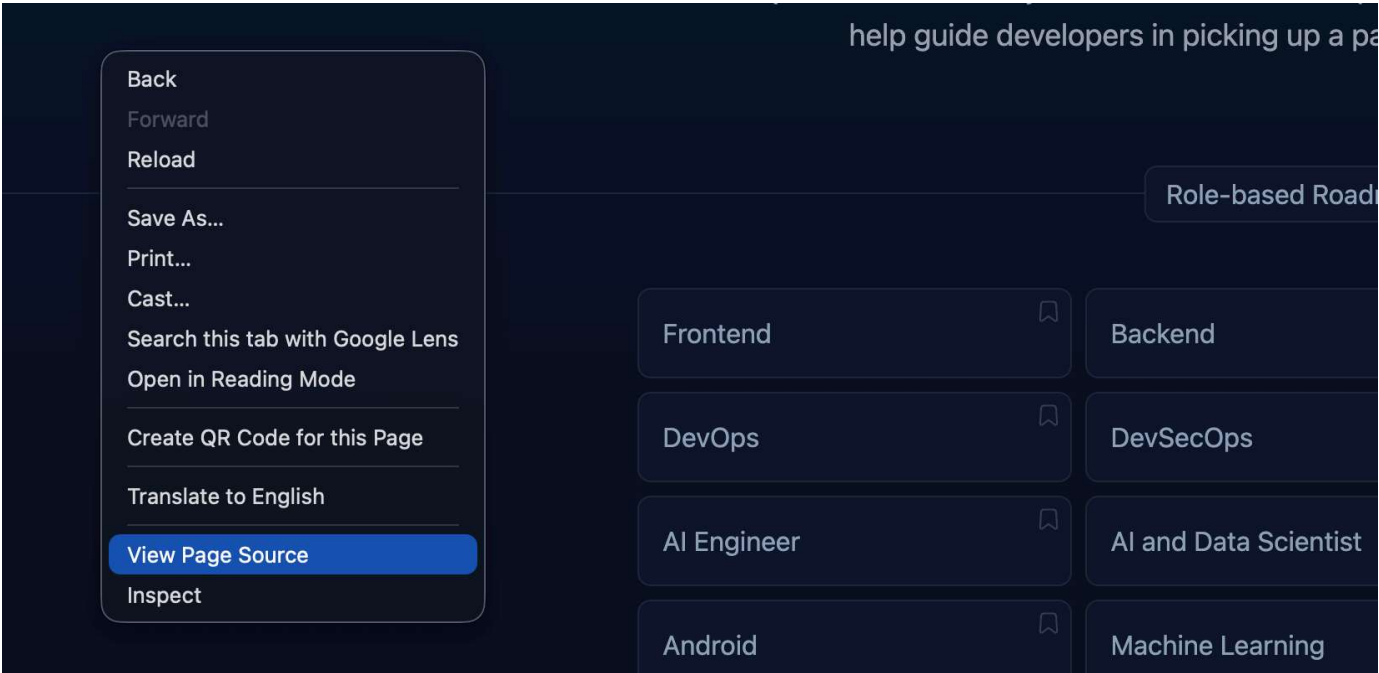
```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>My first page</title>
  </head>
  <body>
    <h1>Hello, web</h1>
    <p>This is a paragraph of text.</p>
  </body>
</html>
```

Now open it in your browser. Open DevTools with **F12** , **Cmd+Option+I** , or by right-clicking and choosing **Inspect**.

Click the **Elements** tab. The tree you see there is the live DOM. You can expand and collapse nodes, hover over them to see which part of the page they correspond to, and watch them update if the page changes.



Now, on the same page, right-click anywhere and choose **View Page Source** (or **View Source**). A new tab opens with the raw HTML source. Compare it with what you saw in Elements.



For this simple file, they should look almost identical. After that, try the same thing on a modern web app. View Source and Elements may look very different there. That difference is the DOM at work.

< 0 / 6 >

Knew

0 Items

Learnt

0 Items

Skipped

0 Items

ResetProgress

Test Your Knowledge

What is HTML?

[Click to Reveal the Answer](#)

- ✔ Already Know that
- ✦ Didn't Know that
- ▶ Skip Question



LESSON COMPLETE

Nicely done.

Up next: Pricing Comparison Table

Next Lesson →